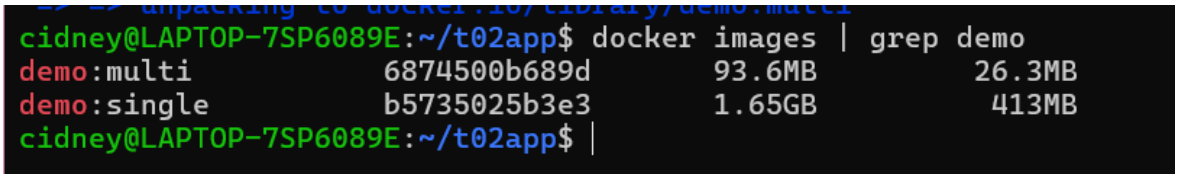
Project 2: Technical Rationale

1. **Base Image Selection**
   - The *nginx:alpine* image was used for the final stage of the Dockerfile because the application only needs a web server to serve the HTML file. The Alpine-based image has a smaller size, which reduces the overall size of the final Docker image. Additionally, a multi-stage build was used to separate the application-building process from the final runtime environment. It resulted to applying Node.js during the build stage without including the entire Node.js environment in the final image.
   - Based on the image size comparison performed in T02, the single-stage image was 1.65 GB, while the multi-stage image was only 93.6 MB, making it approximately 17.6 times smaller.



2. **Service Separation**
   - The Nginx web server and Redis cache were configured as separate services in Docker Compose because they have different purposes. Separating the services allows each service to be managed and updated independently. The `redis:7-alpine` image was used because Redis already provides a caching service, without needing to develop one from scratch. This also demonstrates the flexibility of containers between IaaS and PaaS, where developers can manage their own container configurations while using pre-built services for specific functions.

References

Docker, Inc. (n.d.). Control startup and shutdown order in Compose. Docker Docs. https://docs.docker.com/compose/how-tos/startup-order/

Docker, Inc. (n.d.). Multi-stage builds. Docker Docs. https://docs.docker.com/build/building/multi-stage/

Docker, Inc. (n.d.). Nginx Docker official image overview. Docker Hub. https://hub.docker.com/_/nginx

Redis Ltd. (n.d.). Run Redis Open Source on Docker. Redis Docs. https://redis.io/docs/latest/operate/oss_and_stack/install/install-stack/docker/